

number variable for each frame on the call stack. This is another often confusing point about recursion: even though, from the source code, it looks like there is only one `number` variable, remember that because it is a local variable, there is actually a different `number` variable for each function call.

When `countDownAndUp(3)` is called ❶, a frame is created, and that frame's local `number` variable is set to 3. The function prints the `number` variable to the screen ❶. As long as `number` isn't 0, `countDownAndUp()` is recursively called with `number - 1` ❷. When it calls `countDownAndUp(2)` ❸, a new frame is pushed onto the stack, and that frame's local `number` variable is set to 2. Again, the recursive case is reached and calls `countDownAndUp(1)` ❹, which again reaches the recursive case and calls `countDownAndUp(0)`.

This pattern of making consecutive recursive function calls and then returning from the recursive function calls is what causes the countdown of numbers to appear. Once `countDownAndUp(0)` is called, the base case is reached ❺, and no more recursive calls are made. However, this isn't the end of our program! When the base case is reached, the local `number` variable is 0. But when that base case returns, and the frame is popped off the call stack, the frame under it has its own local `number` variable, with the same 1 value it's always had. As the execution returns back to the previous frames in the call stack, the code *after* the recursive call is executed ❻. This is what causes the count up of numbers to appear. Figure 1-9 shows the state of the call stack as `countDownAndUp()` is recursively called and then returns.

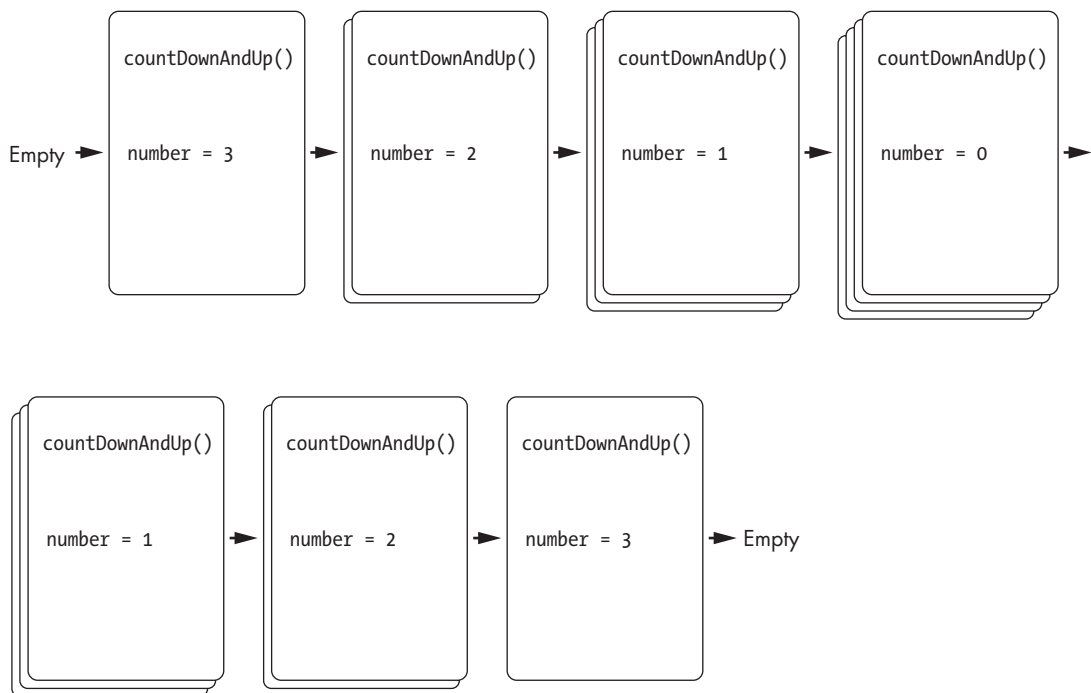


Figure 1-9: The call stack keeping track of the values in the `number` local variable for each function call